



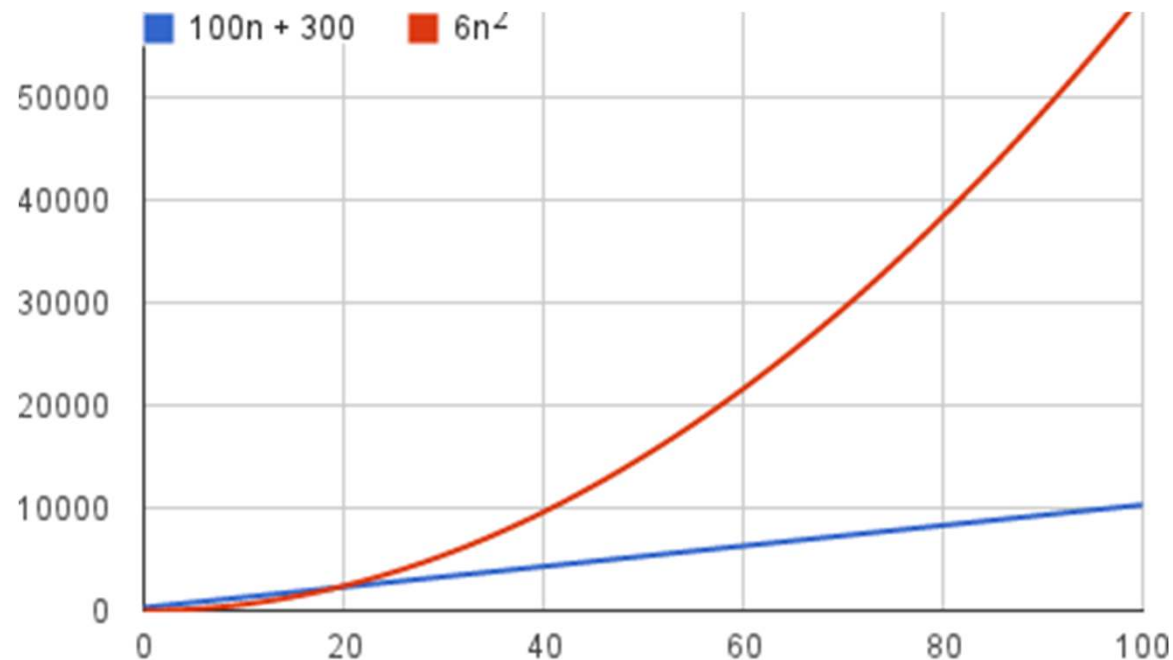
Medidas asintóticas

Estructuras de Datos y Algoritmos I

Profesora: Elba Karen Sáenz García

- FI UNAM

Medidas asintóticas



- Las *cotas de complejidad* o *medidas asintóticas* sirven para clasificar funciones de tal forma que podamos compararlas.
- Permiten analizar qué tan rápido crece el tiempo de ejecución de un algoritmo cuando crece el tamaño de los datos de entrada.

Función	Tipo	n=1	n=5	n =8	n= 10	n=11
$f(n)=n +1$	Pol	2	6	9	11	12
$f(n)=n^2 +n +1$	Pol	3	31	73	111	133
$f(n)=n^3 +n^2 +n +1$	Pol	4	156	585	1111	1464
$f(n)= 2^n$	Exp	2	32	256	1024	2048
$f(n)= n!$	Exp	1	120	40320	3628800	39916800
$f(n)= n^n$	Exp	1	3125	16777216	10 E 10	2,8 E 12

Medidas asintóticas

- Se requiere estudiar el *comportamiento asintótico de los algoritmos, es decir qué sucede con los mismos cuando n tiende a infinito.*

Notaciones

- O (o mayúscula),
- Ω (omega mayúscula) y
- θ (theta mayúscula) y

Orden de Algoritmos

- Es una función $f(n)$ que acota asintóticamente
 - Superior (O)
 - Inferior (Ω)
 - Superior e inferior (Θ)

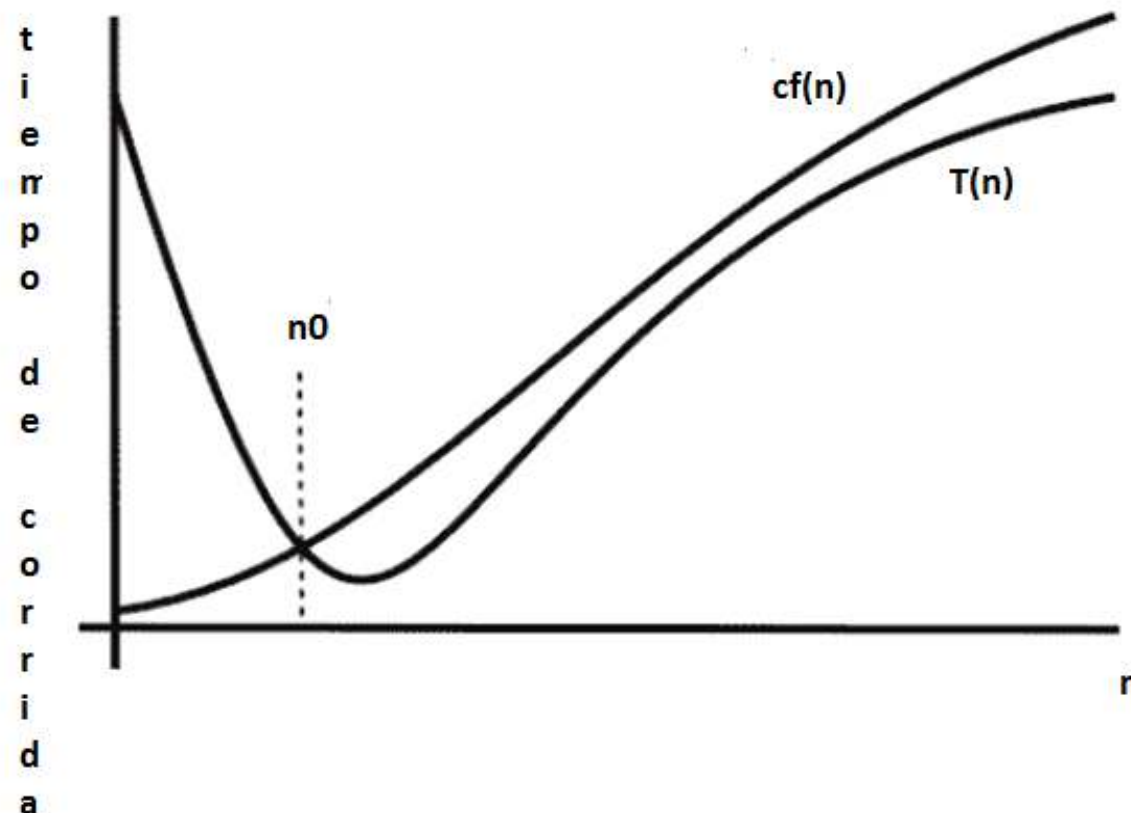
un algoritmo para una entrada determinada, a partir de un valor n_0

La notación O

$O(f(n))$ es el conjunto de todas las funciones T para las cuales existen constantes enteras positivas c y n_0 tales que para $n \geq n_0$ se cumple que:

$$T(n) \leq cf(n)$$

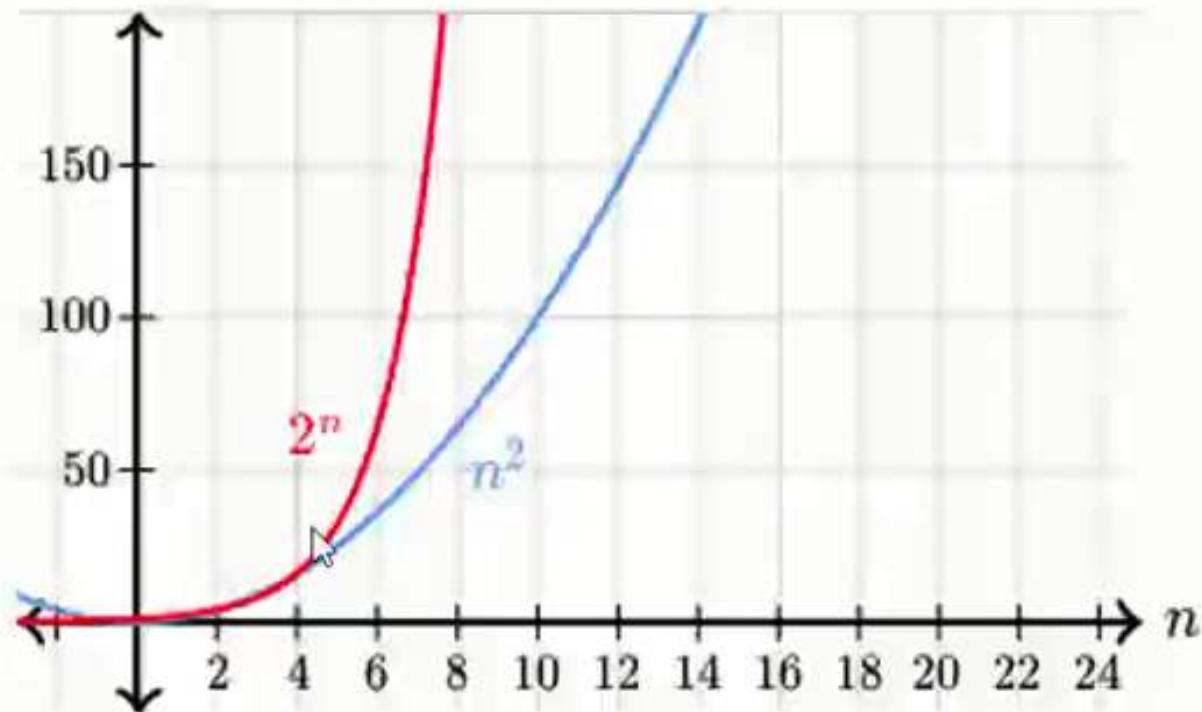
$cf(n)$ es una “cota superior” de toda T para $n \geq n_0$



$$O(f) = \{ t: N \rightarrow R^+ | \exists n_0 \in N, \forall n \geq n_0: t(n) \leq cf(n) \}$$

- si $T(n) \leq cf(n)$ para todo $n > n_0$ implica que $T(n)$ es $\mathbf{O}(f(n))$.
- Cuando se estudia un algoritmo se trata de encontrar la función f mas simple de manera que $T(n)$ pertenezca a $O(f)$ donde T representa el tiempo de ejecución del algoritmo o medida de la memoria necesaria

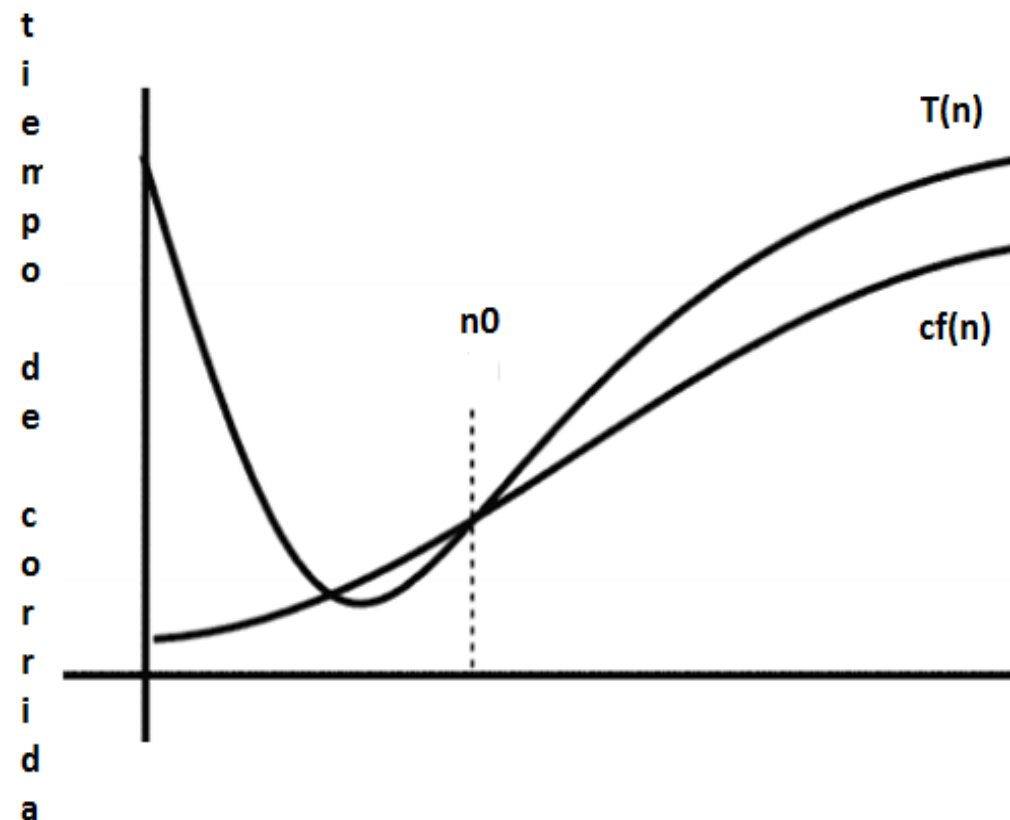
$K \geq 1$ y $c > 1$



n^k es $O(c^n)$

Cota Inferior: Notación omega Ω

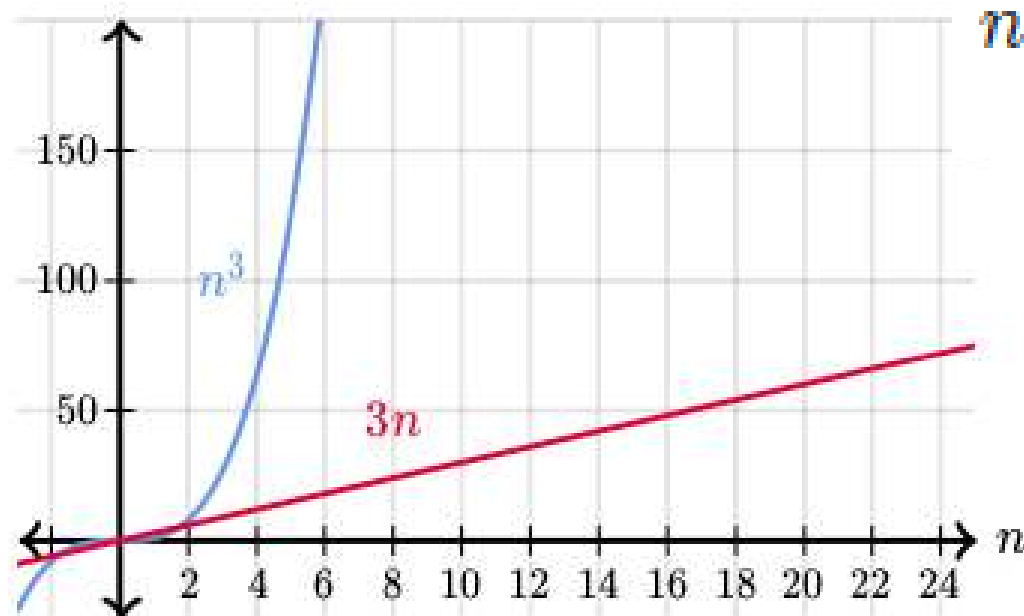
- La notación Ω se utiliza para acotar inferiormente la forma en que crece la función



$$\Omega(f) = \{ t: N \rightarrow R^+ \mid \exists c \in R^+ \exists n_0 \in N, \forall n \geq n_0: t(n) \geq cf(n) \}$$

- ¿Cuál es la relación asintótica entre las funciones

$n^3 \log_2 n$ y $3n \log_8 n$?

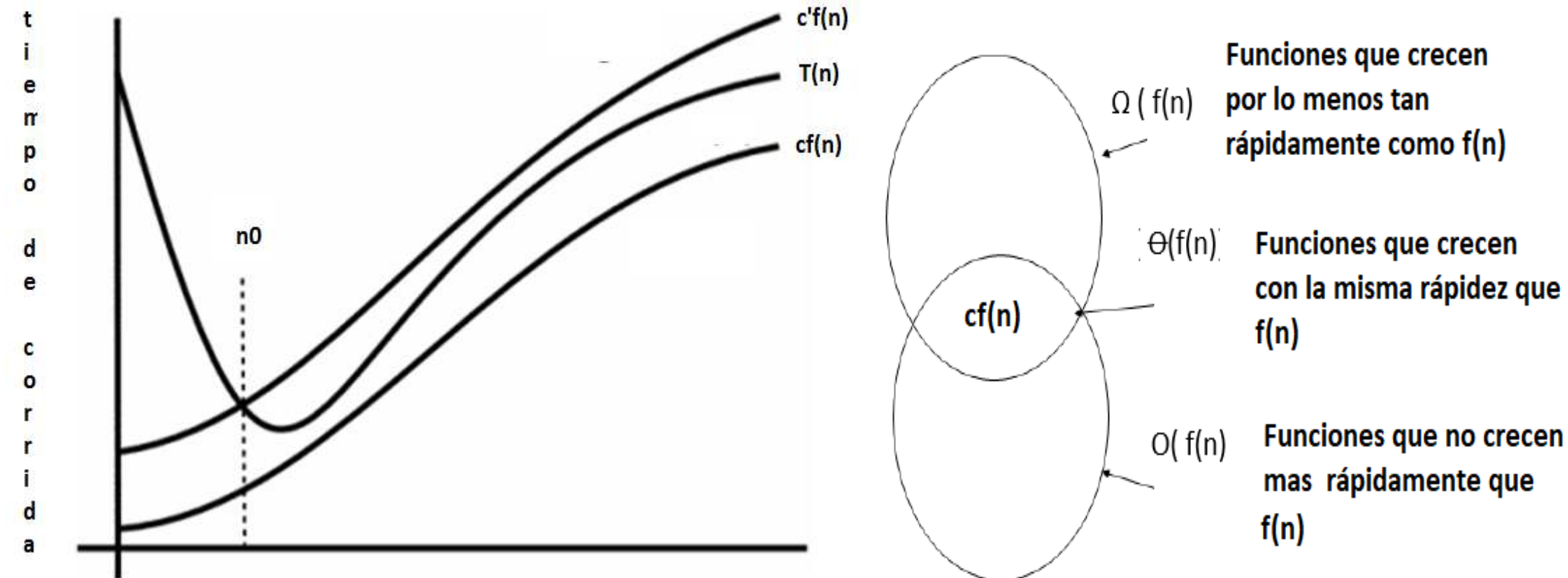


$n^3 \log_2 n$ es $\Omega(3n \log_8 n)$

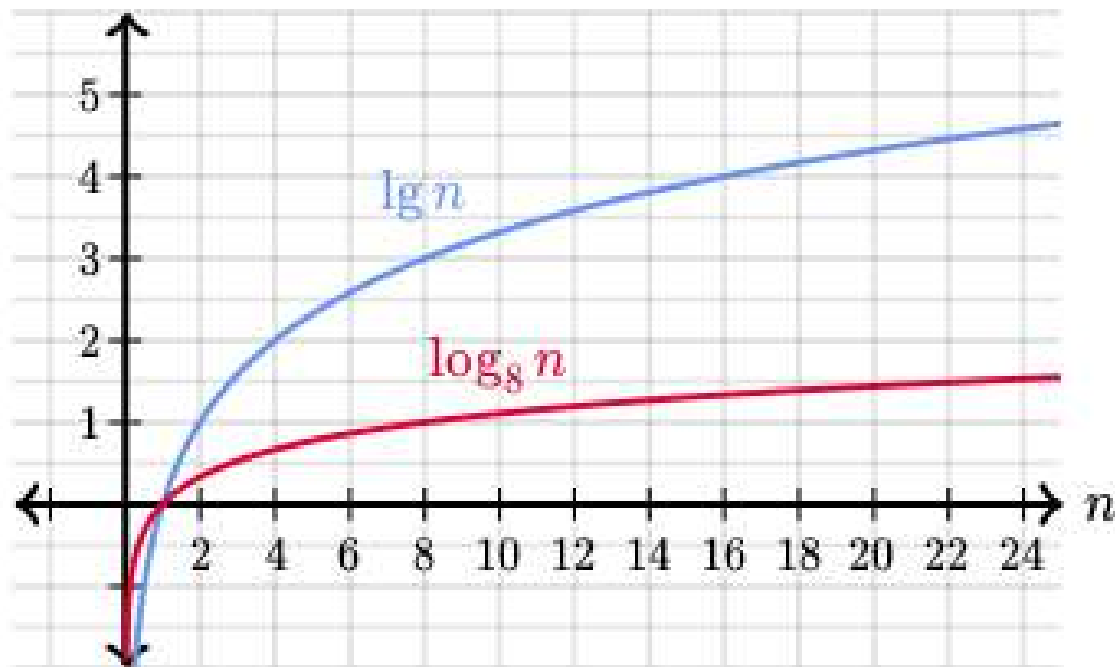
Ejemplo

- Si el tiempo de ejecución $T(n)$ de un programa es $\Omega(n^2)$ implica que:
- $T(n) \geq c n^2$

Cota ajustada asintótica: Notación THETA (Θ) u orden exacto



- ¿Cuál es la relación asintótica entre las funciones $\log_2 n$ y $\log_8 n$?



$$\log_2 n \text{ es } O(\log_8 n)$$

$$\log_2 n \text{ es } \Omega(\log_8 n)$$

$$\log_2 n \text{ es } \Theta(\log_8 n)$$

Algunas funciones $\Theta(g(n))$

- Probar que $1/2n^2 - 3n = \Theta(n^2)$.
 - $c_1 n^2 \leq 1/2n^2 - 3n \leq c_2 n^2$
 - $c_1 \leq 1/2 - 3/n \leq c_2$
 - $c_2 \geq 1/2$ para cualquier $n \geq 1$, y $c_1 \leq 1/14$ para cualquier $n \geq 7$
 - Así, $c_1 = 1/14$, $c_2 = 1/2$, $n_0 = 7$.

Probar que $1/2n^2 - 3n = \Theta(n^2)$.

$$- c_1 n^2 \leq 1/2n^2 - 3n \leq c_2 n^2$$

$$- c_1 \leq 1/2 - 3/n \leq c_2$$

– $c_2 \geq 1/2$ para cualquier $n \geq 1$, y $c_1 \leq 1/14$ para cualquier $n \geq 7$

– Así, $c_1 = 1/14$, $c_2 = 1/2$, $n_0 = 7$.

$$\frac{1}{2} - \frac{3}{n_0} \leq c_2$$

- Probar que $1/2n^2 - 3n = \Theta(n^2)$.
 - $c_1 n^2 \leq 1/2n^2 - 3n \leq c_2 n^2$
 - $c_1 \leq 1/2 - 3/n \leq c_2$
 - $c_2 \geq 1/2$ para cualquier $n \geq 1$, y $c_1 \leq 1/14$ para cualquier $n \geq 7$
 - Así, $c_1 = 1/14$, $c_2 = 1/2$, $n_0 = 7$.

$$\frac{1}{14} \leq \frac{1}{2} - \frac{3}{7}$$

Probar que $6n^3 \neq \Theta(n^2)$.

$$C_2 n^2 \geq 6n^3$$

$$C_2 \geq 6n$$

$$n \leq \frac{C_2}{6}$$

Relación entre $\Theta(g(n))$ y $O(g(n))$

- $f(n) = \Theta(g(n))$ implica que $f(n) = O(g(n))$
- Notación Θ es un concepto más fuerte que Notación O .
- $\Theta(g(n)) \subseteq O(g(n))$
- $an^2+bn+c = \Theta(n^2)$, también $=O(n^2)$

- ¿Cómo encontrar la complejidad del algoritmo?
 - 1) Se puede obtener $T(n)$ y de allí $O(f(n))$
 - 2) Se puede estimar directamente $O(f(n))$

Ejemplo

- Suponer que un algoritmo resuelve un problema en operaciones elementales. $7n^3 + 5n^2 + 27$
- Nos interesa la tasa de crecimiento
- Las diferencias entre $7n^3$ y n^3 no son realmente importantes.
- Se descartan los términos de orden bajo, ya que es el grado mayor quien determina la tasa de crecimiento.

Ejemplo continuación

- El resultado final es que la complejidad de este algoritmo puede ser descrito como n^3

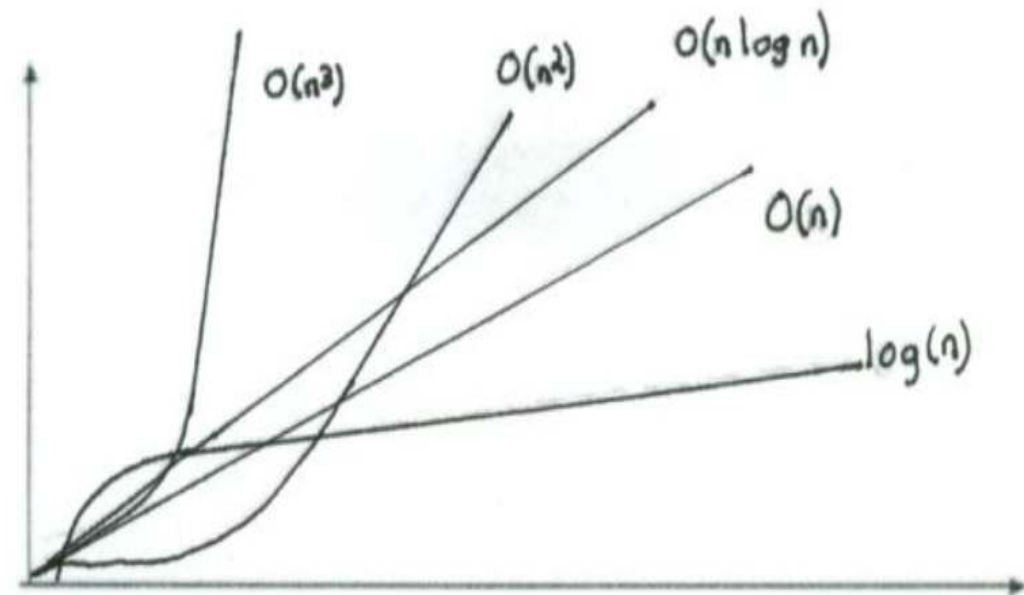
Formalmente se dice que el algoritmo es de orden $O(n^3)$

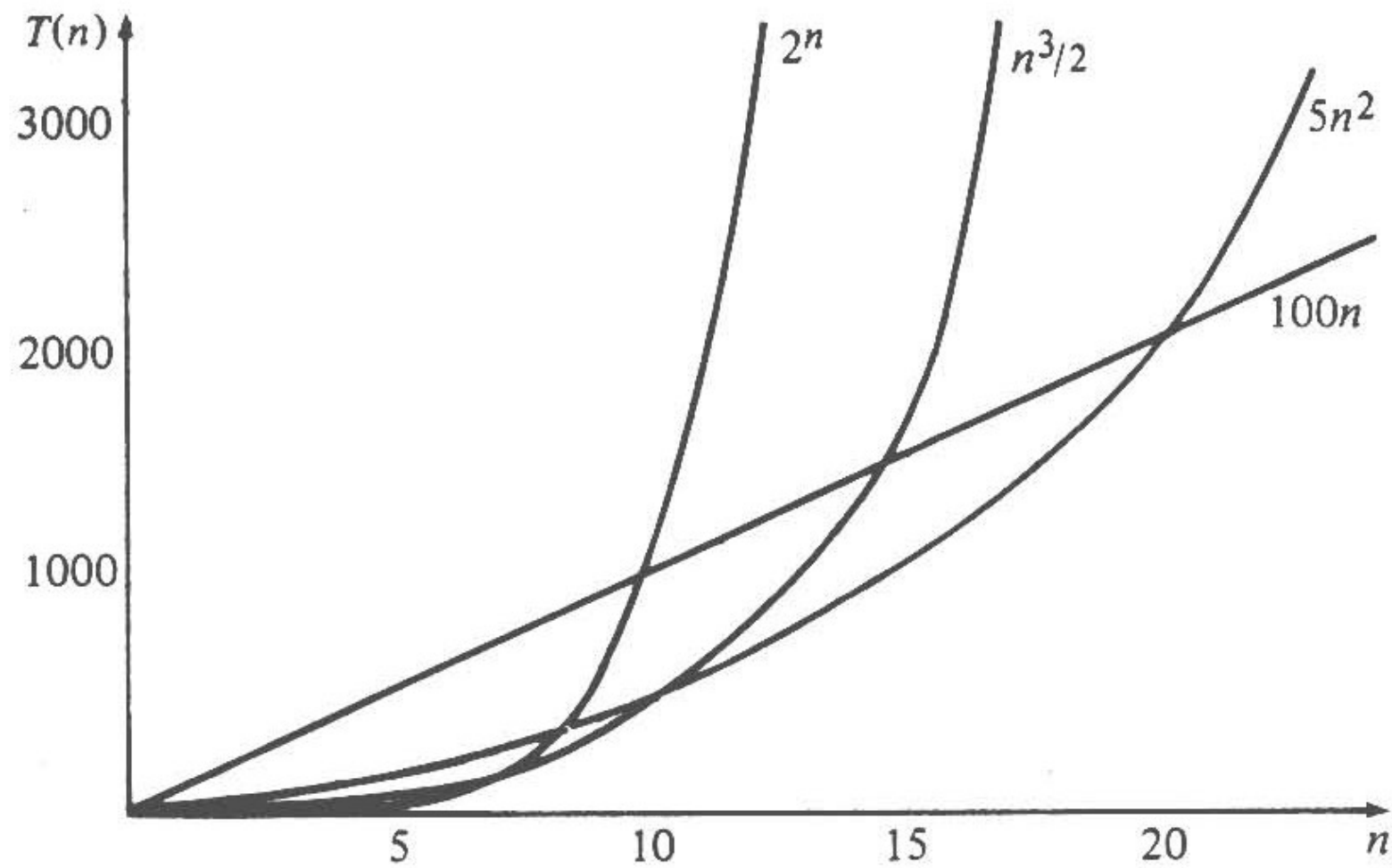
Es también usual decir que este algoritmo “toma un tiempo $O(n^3)$ ”

Ordenes de Complejidad

- *$O(f(n))$ define un orden de complejidad, es decir una “familia” de funciones que crecen de una determinada manera. Así tenemos los siguientes órdenes de complejidad.*

- $O(1)$ Orden constante
- $O(\log n)$ Orden logarítmico
- $O(n)$ Orden lineal
- $O(n \log n)$
- $O(n^2)$ Orden cuadrático
- $O(n^a)$ Orden polinomial ($a > 2$)
- $O(a^n)$ Orden exponencial ($a > 2$)
- $O(n!)$ Orden factorial





Reglas practicas para determinar tiempos y orden de algoritmos

- Si bien no existe una receta para encontrar la O más descriptiva de un algoritmo, muchas veces se pueden aplicar las siguientes reglas.

Instrucciones Simples

- Si $T(n)=c \Rightarrow$ **$T(n)$ es $O(1)$**
- Considerar que el **tiempo de una OE** es, por definición, de orden constante **$O(1)$** .

Composición de Instrucciones

- Si las instrucciones I_1 e I_2 tienen $T_1(n)$ y $T_2(n)$ respectivamente entonces el tiempo de ambas es $T_{12}(n) = T_1(n) + T_2(n)$
- Aplicando la regla de la suma es el máximo de ambos
 $O(T_{12}) = O(\max(T_1, T_2))$

Ciclos

- El tiempo de ejecución de un ciclo es, a lo más, el tiempo de ejecución de las instrucciones que están en el interior del ciclo por el número de iteraciones:

```
for( int i=1; i<=n; i++ )
```

```
    suma = suma+i;           Tinterior(n)
```

```
return suma;
```

n iteraciones

$T(n)=n \cdot T_{\text{interior}}(n)$

Ciclos anidados

- Analizarlos de adentro hacia fuera. Para calcular el tiempo de ejecución total de ciclos anidados se sigue la misma regla de los ciclos.

Complejidad en ciclos

- Se deben distinguir dos casos diferentes.
 - Cuando ***n*** *NO tiene que ver con la cantidad de veces que se ejecuta el bucle*, la repetición introduce una constante multiplicativa que termina absorbiéndose.
 - Cuando ***n*** *determina de alguna manera la cantidad de iteraciones*, sí modificará la complejidad. Veamos algunos ejemplos.

for (i= 0 to **K**) { TinstrucFor(n)=k1 T(n)=k*k1= cte
 algo_de_O(1) => $K * O(1) = O(1)$
 }

for (i= 0 to n) { TinstrucFor(n)=k1 T(n)=n*k1
 algo_de_O(1) => $n * O(1) = O(n)$
 }

for (i= 0 to n) do TinstrucForint(n)=n*k1
 for (j= 0 to n) do TinstrucForExt(n)= n*(n*k1)= k1n^2
 algo_de_O(1) => $n * n * O(1) = n * O(n) = O(n * n) = O(n^2)$

Ejemplo

```
for (int i= 0; i < n; i++) {  
    for (int j= 0; j < i; j++) {  
        algo_de_O(1) } }
```

el ciclo exterior se realiza **n** veces, en cada iteración, el ciclo interior se realiza primero 1 vez , luego 2, 3, ... Y la última n veces.

Entonces

$$T(n) = 1 + 2 + 3 + \dots + n = n*(n+1)/2 \rightarrow O(n^2)$$

Ejemplo

$c=n$

```
while (c > 1) {
    algo_de_O(1)
    c = c/2;
}
```

Se realizan **$\log n$** iteraciones

$\log n * O(1) = O(\log n)$

c	iteracion
n	1
n/4	2
n/8	3
n/16	4
$n/2^k=1$	k

$$\frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n = \log n$$

```
for (int i= 0; i < n; i++) {  
  c= i;  
  while (c > 1) {  
    algo_de_O(1)  
    c= c/2; }  
}
```

- Se tiene un ciclo interno de orden $O(\log n)$ que se ejecuta n .
- Luego el conjunto es de orden:
 $n * O(\log n) = O(n \log n)$ cuasi lineal

Instrucciones de selección

- El tiempo de ejecución de una instrucción condicional “if” es el costo de las instrucciones que se ejecutan condicionalmente, más el tiempo para evaluar la condición.
- . La condición suele ser de orden $O(1)$

```

if(n%2 == 0){  O(1)
  for( i=1; i<= n; i++)  =>  O(1)+n*O(1)=O(1)+O(n)
    x++;    O(1)
}

```

$$T(n) = 2 + \left(1 + 1 + \left(\sum_{i=1}^n 2 + 2 + 1 \right) \right) = 4 + 5n \in \mathbf{O}(n)$$

Funciones

- Se obtiene con el tiempo que se emplea en obtener el valor de cada parámetro, $P_2, \dots, P_n$, más el tiempo que tarde en ejecutarse F , mas la llamada
- $T = 1 + T(P_1) + T(P_2) + \dots + T(P_n) + T(F)$.

Subprogramas(funciones)

- El costo u orden de un subprograma se deriva de las reglas anteriores. Si el subprograma es recursivo entonces hay que distinguir entre los casos base y recurrentes.

Complejidad Espacial

- Memoria que utiliza un programa para su ejecución. La eficiencia en memoria de un algoritmo indica la cantidad de espacio requerido para ejecutar el algoritmo; es decir el espacio en memoria que ocupan todas las variables propias del algoritmo